॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

CS101C: Introduction to Programming (Using C)

Instructors: Prof. Aikta Arya and Prof. Konjengbam Anand

CSE Department

Indian Institute of Technology, Dharwad

Extended Course Essesntials

Today's Class (30/10/2026)

Imagine you are maintaining a song playlist!



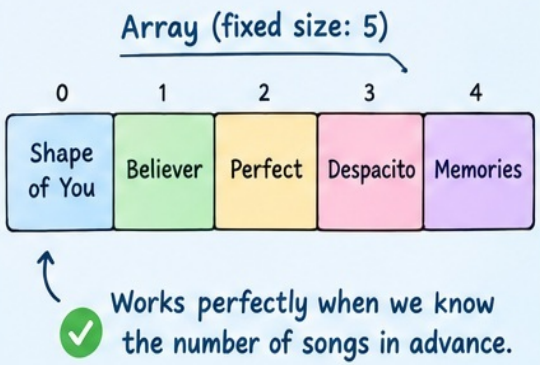
Imagine you are maintaining a song playlist!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad



One possibility is an array.

```
char songs[5][100] = {
  "Shape of You",
  "Believer",
  "Perfect",
  "Despacito",
  "Memories"
};
```



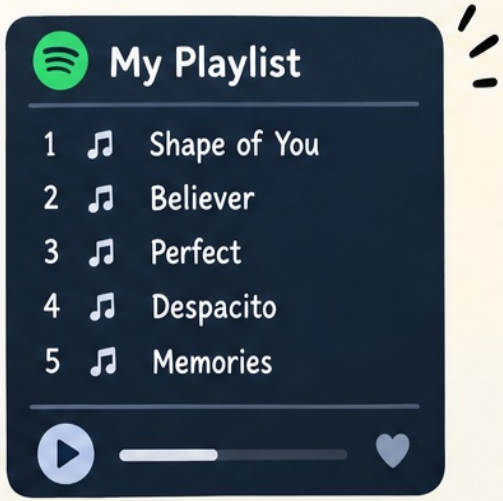
Works perfectly when we know the number of songs in advance.

Imagine you are maintaining a song playlist!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

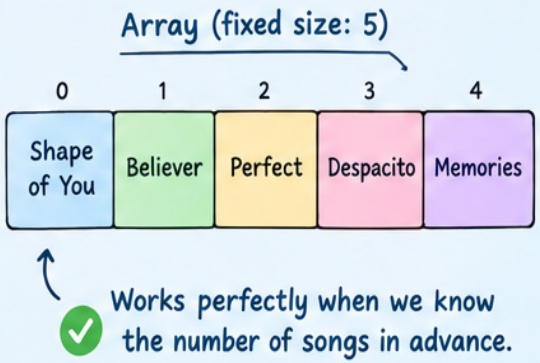
Imagine you are maintaining a playlist!





One possibility is an array.

```
char songs[5][100] = {
  "Shape of You",
  "Believer",
  "Perfect",
  "Despacito",
  "Memories"
};
```



But imagine the following situations:

1 A user adds 100 more songs.



2 A user deletes songs from the middle.



3 A user continuously creates new playlists.



4 We do not know how many songs the user will eventually add.

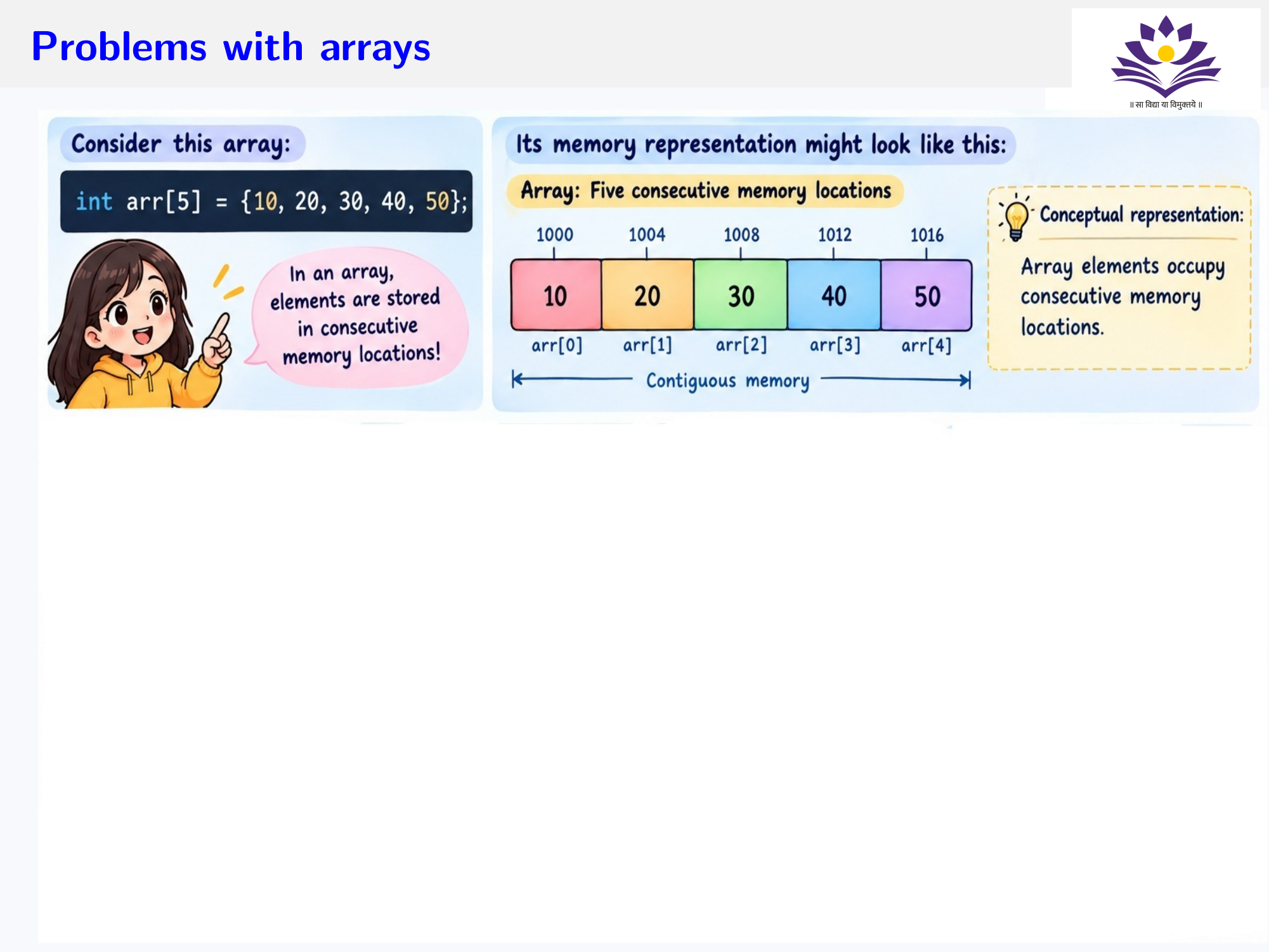


Would an array always be the most convenient choice?

Think about it!



Problems with arrays



Problems with arrays

॥ सा विद्या या विमुक्तये ॥

Consider this array:

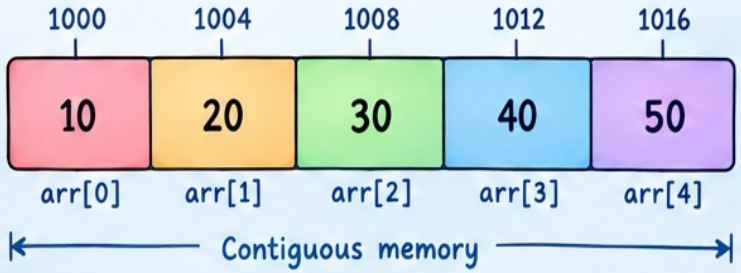
```
int arr[5] = {10, 20, 30, 40, 50};
```



In an array, elements are stored in consecutive memory locations!

Its memory representation might look like this:

Array: Five consecutive memory locations



Conceptual representation:

Array elements occupy consecutive memory locations.



We need another way to organize data!



Problems with arrays

॥ सा विद्या या विमुक्तये ॥

Consider this array:

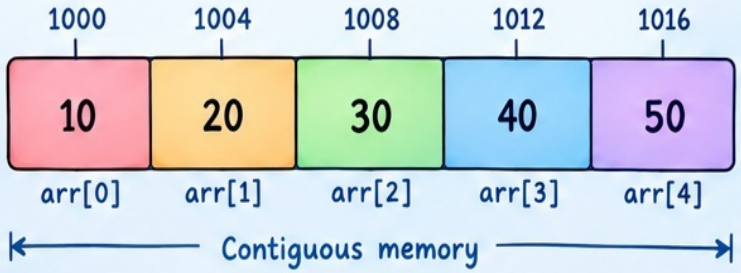
```
int arr[5] = {10, 20, 30, 40, 50};
```



In an array, elements are stored in consecutive memory locations!

Its memory representation might look like this:

Array: Five consecutive memory locations

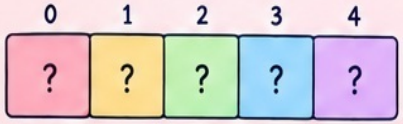


Conceptual representation:

Array elements occupy consecutive memory locations.

1 Fixed size

```
int arr[5];
```



We have space for only five integers.



What if we need more space?



We need another way to organize data!

Let's explore linked lists!



Problems with arrays

॥ सा विद्या या विमुक्तये ॥

Consider this array:

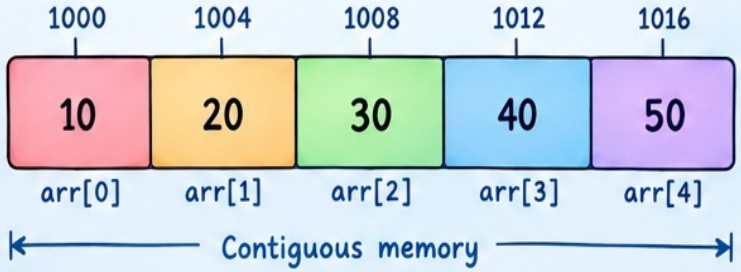
```
int arr[5] = {10, 20, 30, 40, 50};
```



In an array, elements are stored in consecutive memory locations!

Its memory representation might look like this:

Array: Five consecutive memory locations

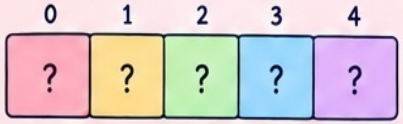


Conceptual representation:

Array elements occupy consecutive memory locations.

1 Fixed size

```
int arr[5];
```



We have space for only five integers.

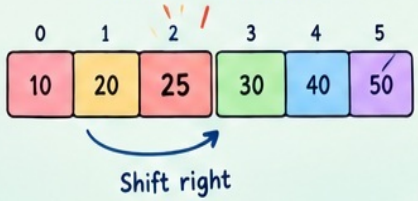


2 Insertion

Suppose we want to insert 25 between 20 and 30.



We may need to shift several elements.





We need another way to organize data!



Problems with arrays

॥ सा विद्या या विमुक्तये ॥

Consider this array:

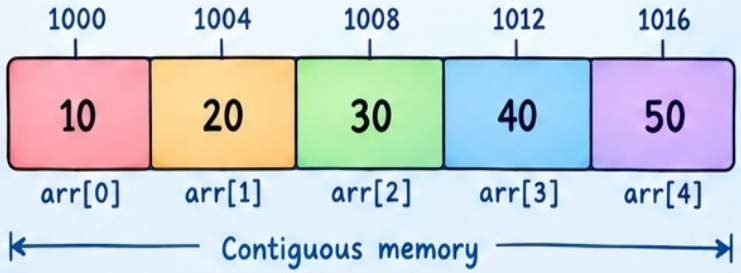
```
int arr[5] = {10, 20, 30, 40, 50};
```



In an array, elements are stored in consecutive memory locations!

Its memory representation might look like this:

Array: Five consecutive memory locations

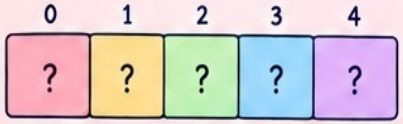


Conceptual representation:

Array elements occupy consecutive memory locations.

1 Fixed size

```
int arr[5];
```



We have space for only five integers.

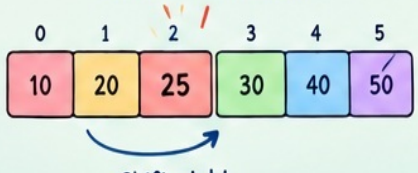


2 Insertion

Suppose we want to insert 25 between 20 and 30.



We may need to shift several elements.

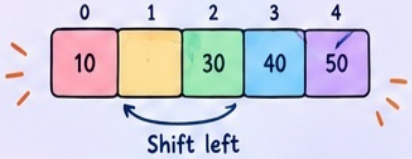


3 Deletion

Suppose we delete 20.



We may need to shift the remaining elements to fill the gap.





We need another way to organize data!



Problems with arrays

॥ सा विद्या या विमुक्तये ॥

Consider this array:

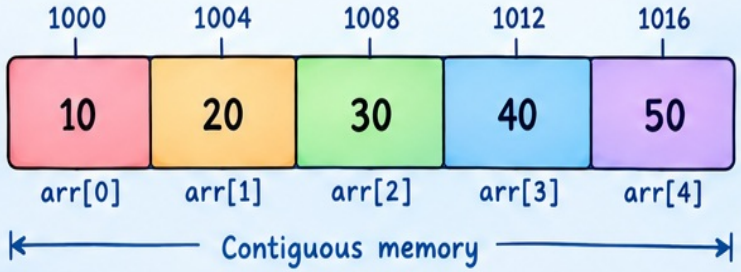
```
int arr[5] = {10, 20, 30, 40, 50};
```



In an array, elements are stored in consecutive memory locations!

Its memory representation might look like this:

Array: Five consecutive memory locations

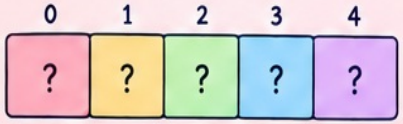


Conceptual representation:

Array elements occupy consecutive memory locations.

1 Fixed size

```
int arr[5];
```

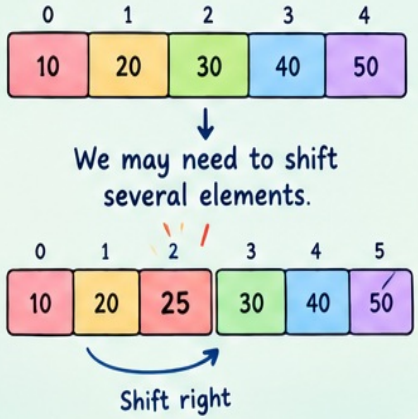


We have space for only five integers.



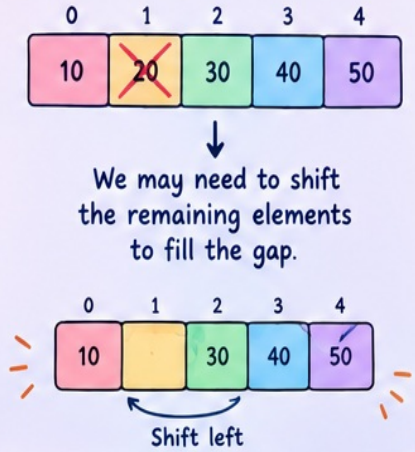
2 Insertion

Suppose we want to insert 25 between 20 and 30.



3 Deletion

Suppose we delete 20.



4 Memory allocation

An array generally requires a contiguous block of memory.

What if sufficient contiguous memory is not available, even though several smaller free memory blocks exist?



No single contiguous block is large enough!



We need another way to organize data!

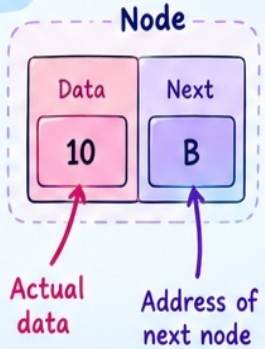


A different way of storing data

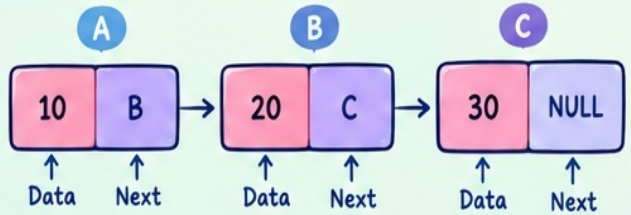


Each box (node) contains:

1. The actual data.
2. The address of the next box.



Our playlist in memory



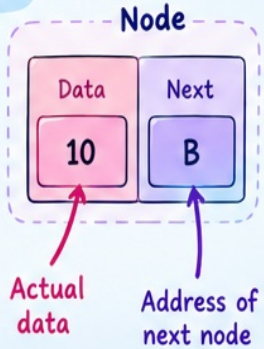
Each box knows where the next box is located. The boxes need not be adjacent in memory.

A different way of storing data

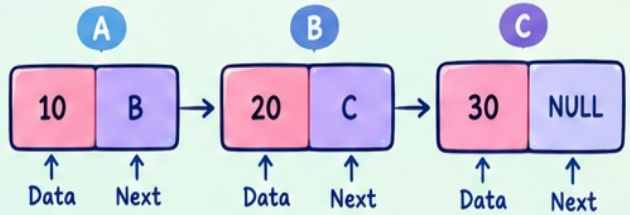


Each box (node) contains:

1. The actual data.
2. The address of the next box.

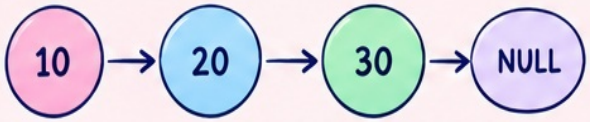


Our playlist in memory

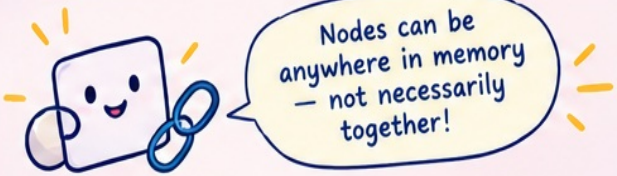


Each box knows where the next box is located. The boxes need not be adjacent in memory.

This is the basic idea behind a Linked List.



A linked list is a collection of nodes where each node stores data and a pointer to another node.



Slide 4. Array vs Linked List

Feature	Array	Linked List
Memory	Contiguous	Can be non-contiguous
Size	Usually fixed after creation	Can grow or shrink dynamically
Access	Direct indexing, O(1)	Sequential traversal, O(n)
Insertion	May require shifting	O(1) if insertion position is already known
Deletion	May require shifting	O(1) if node and required links are known
Extra memory	No link pointer	Requires pointer(s)

Important: Linked lists are not always better than arrays.
They solve different problems and have different trade-offs.



What is a node

A linked list consists of individual elements called **nodes.**



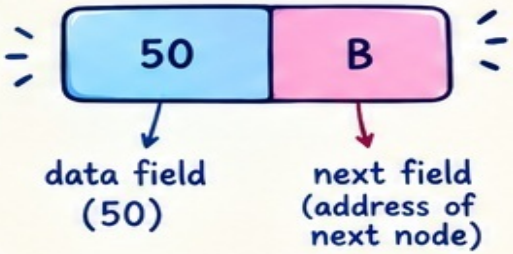
Every node in a singly linked list has two parts:

One Node

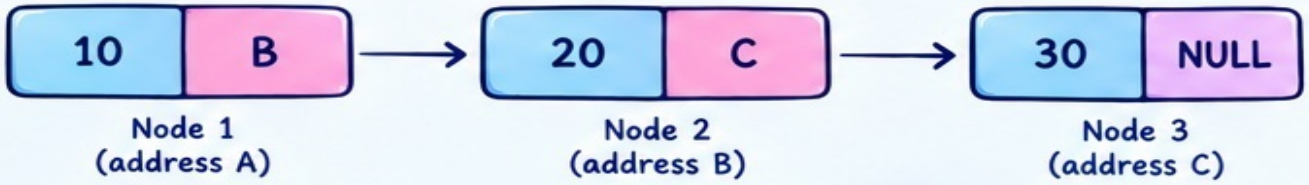


Example:

If a node stores the integer 50, its data field contains 50, while its next field contains the address of the next node.



In memory, it may look like this:



Each node knows where the next node is located.
The boxes need not be adjacent in memory.



This is the basic idea behind a Linked List.

Representing a node in C

॥ सा विद्या या विमुक्तये ॥
य प्रौद्योगिकी संस्थान धारवाड
itute of Technology Dharwad

We already know how to create structures!

```
struct student {
    int roll;
    float marks;
};
```

This is a structure for a student.



Similarly, we can create a structure for a linked-list node.

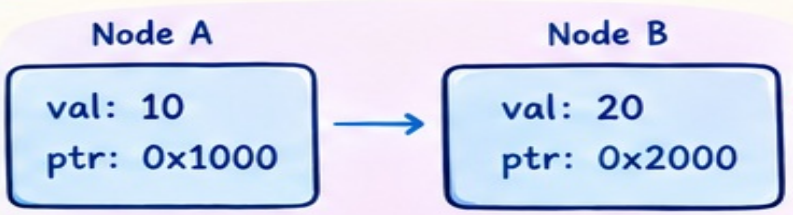
```
struct node {
    int val;
    struct node *ptr;
};
```

This structure has two members:

1. **int val**
   Stores the actual data.
2. **struct node *ptr**
   Stores the address of another structure variable of type struct node.

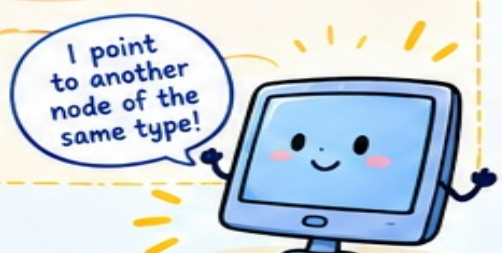
Let's understand this carefully.

Wait! Something interesting is happening!

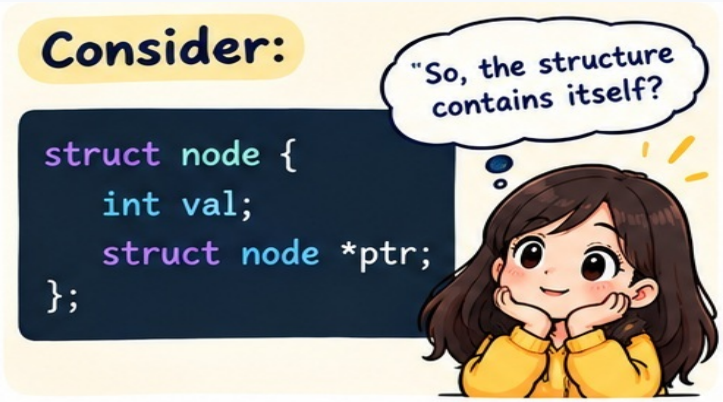


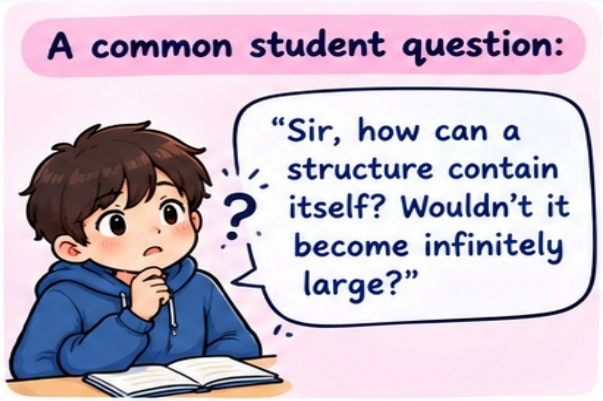
A structure contains a pointer that points to another structure of the same type.

This is called a self-referential structure.



What is a self-referential structure?





What is a self-referential structure?

Consider:

```
struct node {
    int val;
    struct node *ptr;
};
```



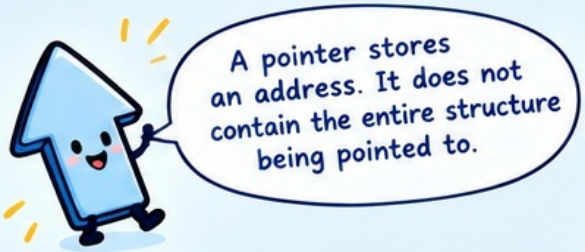
A common student question:



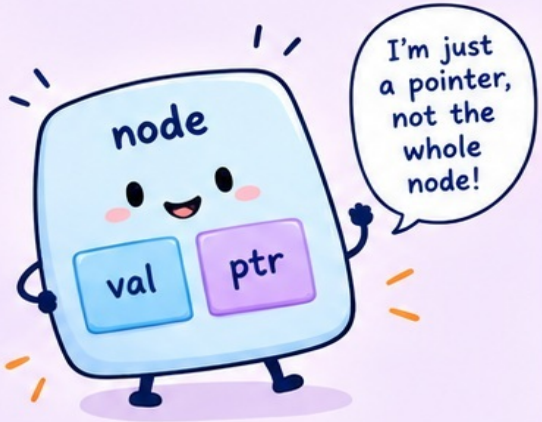
struct node *ptr;

We are **NOT** declaring another complete structure inside the structure.

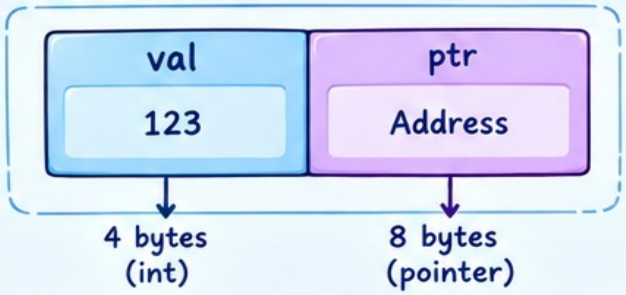
We are only declaring a **pointer variable**.



Therefore, the structure has a **finite size.**



Memory layout of one node



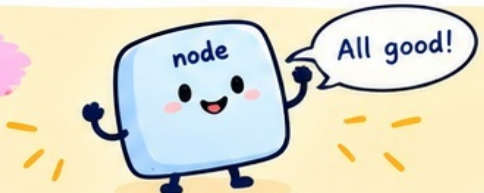
For a typical 64-bit system with a 4-byte int and 8-byte pointer, sizeof(struct node) is commonly 16 bytes because of padding.

Actual size is implementation-dependent.

In short:

A self-referential structure is perfectly valid!

It uses a pointer, not infinite memory.



Creating our first structure variable

Consider:

```
struct node {
    int val;
    struct node *ptr;
};

int main() {
    struct node n1;
}
```



Suppose its starting address is 0x100.



Conceptually:

Variable	Address	Value
n1.val	0x100	Uninitialized
n1.ptr	0x108 (illustriative)	Uninitialized

Remember: the exact addresses and field offsets depend on the system and structure padding.

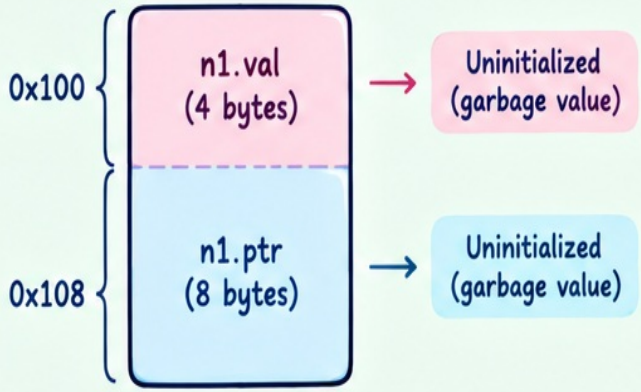
What does this mean?

1. We have allocated memory for the structure variable `n1`.
2. The fields `val` and `ptr` are there, but they are not initialized yet.

They contain garbage values (for now).



Memory view (illustrative)



Size depends on system and padding.

Key Takeaways

- `n1` is a structure variable of type `struct node`.
- It occupies a block of memory.
- Its fields can be accessed using `n1.val` and `n1.ptr`.
- The values are uninitialized until we assign them.

Next, we'll see how to actually use this node!



Writing data inside the node

```
struct node n1;
n1.val = 123;
```

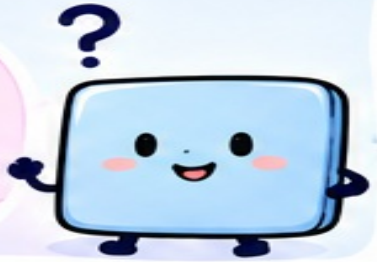


How do we access the pointer field?

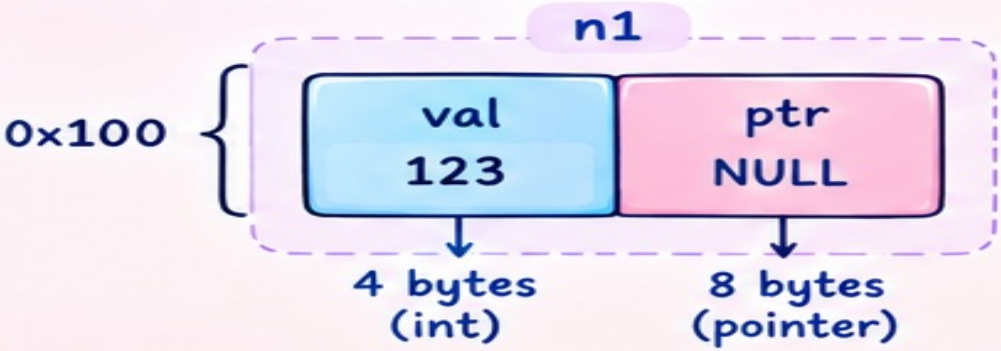
```
n1.ptr = NULL;
```

Why NULL?

Because currently, we have not connected n1 to any other node.



Memory view (illustrative)



We have written 123 to the data field and NULL to the pointer field.

So far, we have created one node and initialized its data and pointer fields.



Creating a second node

```
struct node n1, n2;
n1.val = 123;
n2.val = 456;
```

Suppose:
- n1 is at 0x100
- n2 is at 0x10C

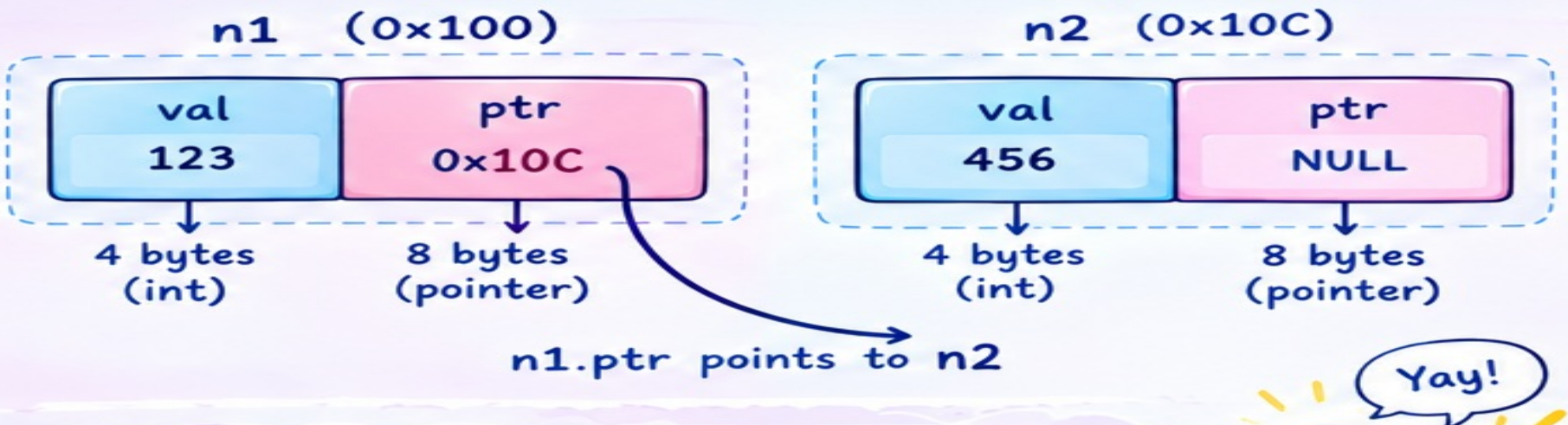



```
n1.ptr = &n2;
```

This is the most important statement!

We are storing the address of n2 inside the pointer field of n1.

Memory view (illustrative)



Now we have connected two structures!

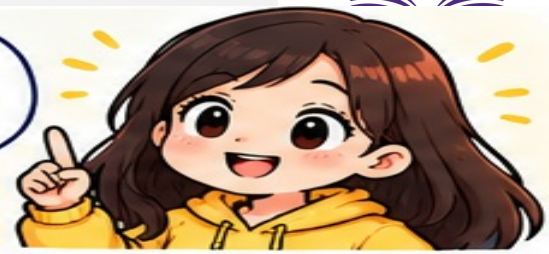


Accessing the next node

We know:

```
n1.ptr = &n2;
```

Now, we want to access the value stored in n2.



We can write:

```
(*n1.ptr).val
```

Or, using the arrow operator:

```
n1.ptr->val
```

Both mean exactly the same thing!

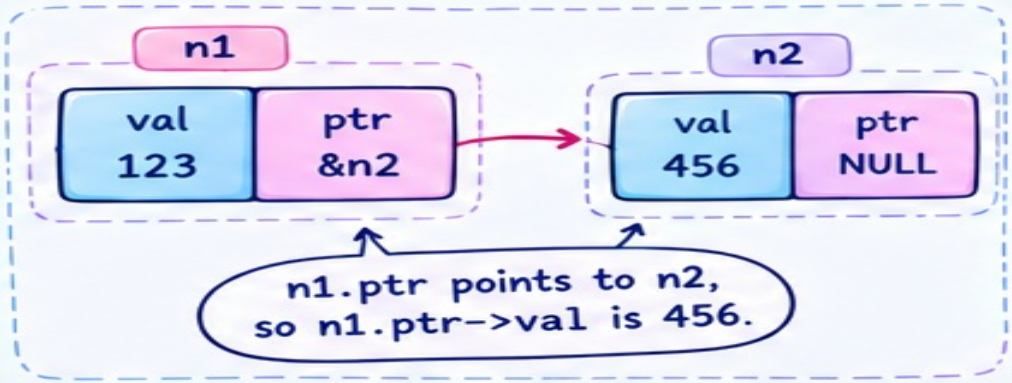


Let's understand the arrow operator:

```
n1.ptr->val
```

It means:

1. Follow the address stored in n1.ptr.
2. Reach the structure at that address.
3. Access its `val` member.



Golden rule:

`ptr->val` is equivalent to `(*ptr).val`

The parentheses are important because `.` has higher precedence than `*`.



Connecting three nodes

Now consider:

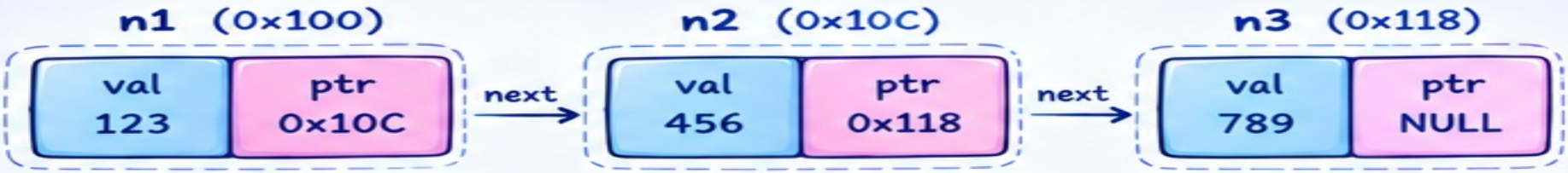
```
struct node n1, n2, n3;

n1.val = 123;
n2.val = 456;
n3.val = 789;

n1.ptr = &n2;
n2.ptr = &n3;
n3.ptr = NULL;
```



The linked list looks like this:



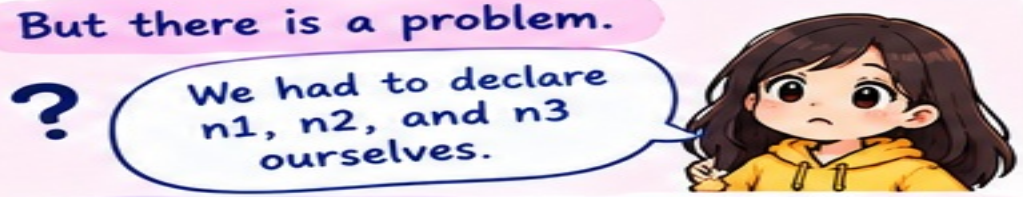
Three nodes connected through their pointer fields.

Congratulations!

We have manually created a linked list!

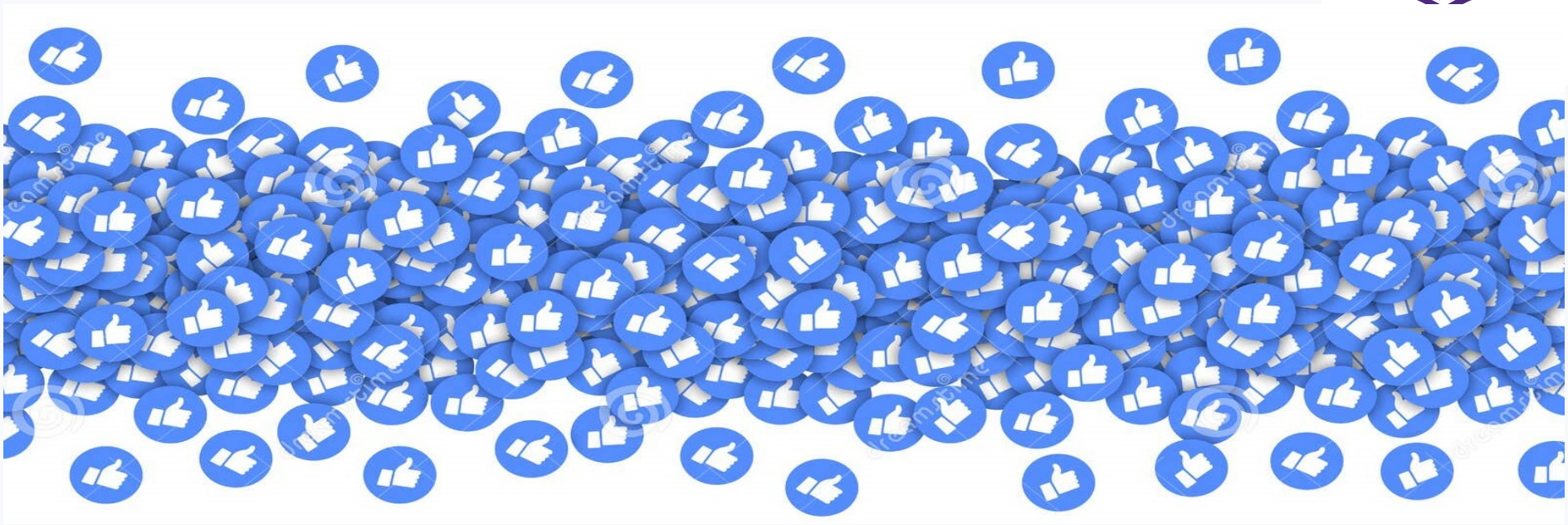


But there is a problem.



- What if the user wants 100 nodes?
- What if the user wants 10,000 nodes?

We need dynamic memory allocation.



Thank you for your kind attention!!